

Apuntes teorico 27/03

materia: programacion 4

profesor: Antonio

repaso en clase, personalización de terminal con la web:



React: Componentes y JSX

- **Componente**

Es la pieza fundamental en React. Para ser un buen componente, debe ser **reutilizable** (puede usarse en múltiples partes de la app) y **adaptable** (puede cambiar su aspecto o datos mediante propiedades llamadas props).

- **JSX (JavaScript XML)**

Es una extensión de sintaxis que permite escribir código que parece HTML dentro de JavaScript. **No es una sentencia**, sino una **expresión**. Esto significa que puede evaluarse y devolver un valor (en este caso, elementos de la interfaz).

- **Incrustar expresiones en JSX**

Para combinar lógica de JavaScript dentro del JSX, se utilizan **llaves { }**. Dentro de ellas puedes poner variables, funciones o cualquier expresión válida de JavaScript

Expresión vs. Sentencia

- **Expresión:** Cualquier unidad de código que se evalúa y produce un valor (ej: `2 + 2`, `variable`, `miFuncion()`).
- **Sentencia:** Instrucciones que realizan una acción pero no devuelven un valor directamente (ej: `if`, `switch`, `for`).
- **En JSX solo se pueden usar expresiones**, no sentencias. Por eso no podemos poner un `if` o `switch` directamente dentro de las llaves `{}`.

Hooks y Estado

- **Hooks**

Son funciones especiales de React que permiten “engancharse” a sus características (como el estado) desde componentes funcionales.

- **Tres reglas de los Hooks**

1. **Solo llamarlos en el nivel superior:** No se llaman dentro de bucles, condicionales o funciones anidadas.
2. **Solo llamarlos desde funciones de React:** Se usan dentro de componentes funcionales o desde hooks personalizados.
3. El nombre suele empezar con use (ej: useState).

- **useState (El estado)**

Es el hook que permite añadir estado a un componente funcional. El estado es la memoria interna del componente.

- **Destructuración (Destructuring)**

Es una sintaxis de JavaScript que permite extraer valores de un arreglo u objeto en variables individuales.

Ejemplo con useState:

- javascript

// En lugar de hacer esto:

```
const resultado = useState(false);
```

```
const isShow = resultado[0];
```

```
const setIsShow = resultado[1];
```

// Usamos destructuración:

```
const [isShow, setIsShow] = useState(false);
```

- **Handler (Manejador)**

Es una función que se ejecuta en respuesta a una interacción del usuario (como un clic). Se suele nombrar con la palabra handle (ej: handleMostrarTexto).

- **Funciones Flecha (Arrow Functions)**

Es una forma más corta de escribir funciones en JavaScript. Si la función retorna una sola cosa (una línea), se puede omitir el return y las llaves {}.

Ejemplo: const sumar = (a, b) => a + b;

Conceptos de Programación General

- **Tipos de Datos**

- **Primitivos:** Inmutables y se almacenan por valor. Ej: number, string, boolean, null, undefined.
- **No primitivos:** Mutables y se almacenan por referencia. Ej: object, array, function.

- **Callback**

Es una función que se pasa como argumento a otra función, con la idea de que se “llame de vuelta” (se ejecute) más tarde, generalmente después de que una tarea termine.

- **Refactorizar**

Es el proceso de reescribir el código fuente sin cambiar su comportamiento externo, con el objetivo de mejorar su estructura, legibilidad o mantenibilidad.

